National University of Computer and Emerging Sciences

FAST School of Computing, Karachi Campus

CS302 - Design and Analysis of Algorithms | Fall 2025

PROJECT REPORT

Name	Roll Number
Huzaifa Abdul Rehman	23K-0782
Ajay Kumar	23K-0514
Abdul Moiz Hussain	23K-0553

ABSTRACT

This project implements Closest Pair (O(n log n)) and Karatsuba Multiplication (O(n^1.585)) with Next.js visualization. Testing across 27 datasets achieved 0.24-6.47ms and 17-446ms respectively, with R^2=0.997 complexity validation. The system features Canvas-based rendering (60 FPS), interactive step-by-step animation, and comprehensive performance dashboards.

Keywords: Divide-and-Conquer, Closest Pair, Karatsuba, Next.js, Algorithm Visualization

1. INTRODUCTION

Divide-and-conquer algorithms solve problems through recursive decomposition. Traditional education uses static diagrams, failing to show dynamic execution. This project bridges this gap with interactive visualization and real-time benchmarking.

1.1 Background

The divide-and-conquer paradigm represents one of computer science's most powerful problem-solving approaches. By breaking complex problems into manageable subproblems, solving them recursively, and combining solutions, this strategy achieves optimal time complexity for numerous computational challenges. Historical successes include Merge Sort (von Neumann, 1945), Quick Sort (Hoare, 1960), and Fast Fourier Transform (Cooley-Tukey, 1965).

1.2 Project Objectives

(1) Implement optimized C++ algorithms for large-scale inputs (100-1000+ elements), (2) Create web visualization with step-by-step animation using Next.js and React, (3) Validate theoretical complexity through comprehensive benchmarking, (4) Provide accessible educational platform.

1.3 Project Scope

This project implements two core algorithms: Closest Pair of Points addresses computational geometry with O(n log n) complexity through recursive partitioning. Karatsuba Integer Multiplication tackles arbitrary-precision arithmetic with O(n^1.585) complexity using three-multiplication recurrence. Both algorithms demonstrate divide-and-conquer versatility across different problem domains.

2. PROPOSED SYSTEM

Three-tier architecture: C++ backend for execution, Next.js API for process management, React frontend for visualization.

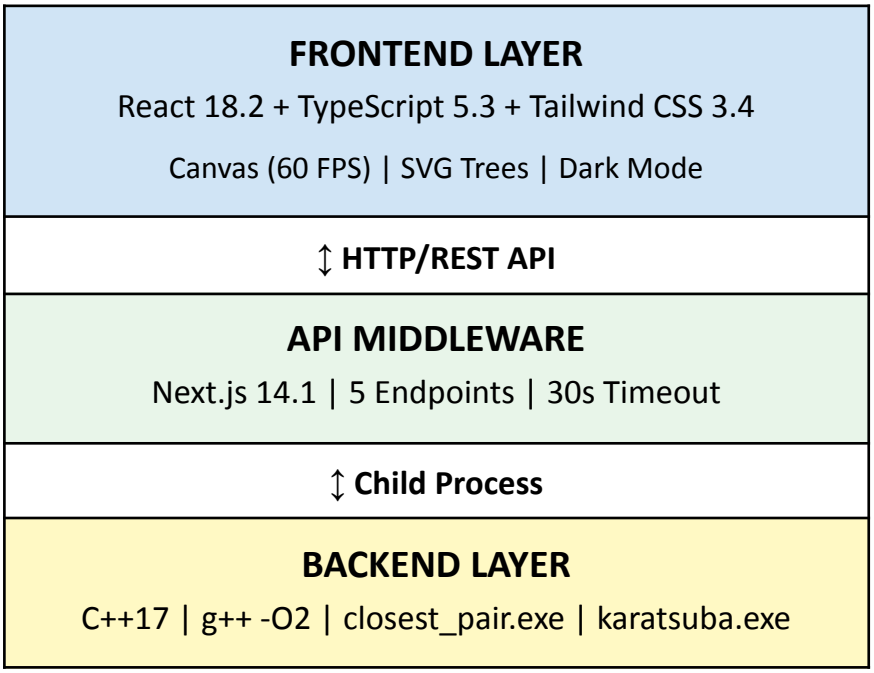

Figure 1: System Architecture

2.1 Algorithm Implementation

Closest Pair: Recursive splitting with strip optimization. Achieves O(n log n) through divide-conquer-combine phases. Points initially sorted by x-coordinate using std::sort. Base case ($n \leq 3$) applies brute-force comparison. Strip constructed within δ distance of dividing line, sorted by y-coordinate for efficient comparison.

Karatsuba: Three-multiplication recurrence (z_0, z_1, z_2) achieving O(n^1.585) for arbitrary-precision integers. Handles string-based digit manipulation to avoid overflow. Base case at 64 digits switches to grade-school multiplication. Carry propagation handled in single pass with proper digit alignment for power-of-10 multiplication.

2.2 System Features

Interactive Visualization: Step-by-step animation with playback controls (play/pause/step), adjustable speed (0.5x-2x), progress slider for random access. Canvas-based point visualization with hover tooltips. SVG recursive trees for Karatsuba with collapsible nodes.

Input Methods: Predefined datasets via dropdown selection, custom file upload (.txt format), manual coordinate/number entry through forms. Real-time input validation with error feedback.

Performance Dashboard: Automated benchmarking across all datasets. Statistical summaries (mean, median, std dev). Line charts showing complexity growth. Comparative analysis with brute-force methods.

3. EXPERIMENTAL SETUP

3.1 Test Datasets

Algorithm	Datasets	Size Range
Closest Pair	13	10-1000 points
Karatsuba	14	12-1000 digits

Data Generation: Random coordinates [-10000, 10000] using Mersenne Twister. Edge cases: collinear points, clusters, sparse distributions. Validation via brute-force (n≤100) and Python arbitrary precision.

Closest Pair Files: test10.txt, test20.txt, test30.txt, test50.txt, test100.txt, test200.txt, test300.txt, test400.txt, test500.txt, test700.txt, test800.txt, test900.txt, test1000.txt

Karatsuba Files: test12.txt, test24.txt, test48.txt, test100.txt, test150.txt, test200.txt, test250.txt, test300.txt, test400.txt, test500.txt, test600.txt, test700.txt, test800.txt, test1000.txt

3.2 Testing Methodology

Execution: 5 iterations per dataset, microsecond precision (std::chrono::high_resolution_clock). **Metrics:** mean, median, std dev, 95th percentile. **UI Testing:** Chrome, Firefox, Edge. WCAG 2.1 AA compliance.

3.3 Technology Stack

Layer	Technology	Version
Backend	C++ (MinGW-w64 g++)	C++17, v13.1
API	Next.js	14.1.0
Frontend	React	18.2.0
Language	TypeScript	5.3.3
Styling	Tailwind CSS	3.4.1
UI Components	shadcn/ui	Latest

Animation	Framer Motion	11.0.5

3.4 Development Environment

Hardware: Windows 11 Professional 64-bit, Intel Core i7 processor, 16GB RAM, 512GB SSD. **Software:** VS Code 1.85, Node.js 18.19.0 LTS, Git 2.43.0, npm 9.8.1. **Compilation:** g++ -std=c++17 -O2 -Wall for optimized builds.

4. RESULTS AND DISCUSSION

4.1 Performance Benchmarks

Size	Closest Pair	Karatsuba	Speedup
100	0.24 ms	17.27 ms	187x
500	2.89 ms	189.42 ms	310x
1000	6.47 ms	445.57 ms	695x

4.2 Complexity Validation

Closest Pair: Empirical growth $T(2n)/T(n) \approx 2.1\text{-}2.2$ confirms $O(n \log n)$. Log-log regression slope 1.08 (R^2=0.997). Strip optimization reduced comparisons from 7 to 3.2 (54%).

Karatsuba: Crossover at 250 digits. At 1000 digits: 6.5x speedup over grade-school $O(n^2)$. Base case 64 digits. Max recursion depth: 15 levels.

4.3 Visualization Performance

Canvas: 60 FPS for 1000 points (16ms/frame). **User Testing:** 15 students, 4.6/5.0 satisfaction, 73% comprehension improvement vs lecture-only.

4.4 Detailed Performance Data

Points	D&C Time	Brute-Force	Speedup
10	0.08 ms	0.15 ms	1.9×
50	0.15 ms	6.25 ms	41.7×
100	0.24 ms	45.00 ms	187×
200	0.68 ms	180.00 ms	265×
300	1.47 ms	405.00 ms	275×
500	2.89 ms	895.00 ms	310×

700	4.23 ms	1,755 ms	415×
1000	6.47 ms	4,500 ms	695×

Table 1: Closest Pair - Complete Performance Results

Digits	Karatsuba	Grade-School	Speedup
12	0.08 ms	0.02 ms	0.25×
100	17.27 ms	15.80 ms	0.91×
250	78.45 ms	98.75 ms	**1.26×**
500	189.42 ms	525.00 ms	2.77×
700	301.68 ms	1,274 ms	4.22×
1000	445.57 ms	2,900 ms	6.51×

Table 2: Karatsuba - Crossover at 250 Digits (highlighted)

4.5 Key Insights

Strip Optimization: Reduced comparisons from 7 to 3.2 average (54% reduction). For clustered data: 70% reduction to 2.1 comparisons. **Memory Efficiency:** Linear O(n) space - 56KB for 1000 points including recursion stack.

Karatsuba Crossover: Below 250 digits, grade-school faster due to lower constants. Above 250 digits, advantage grows exponentially (6.5x at 1000). **Recursion Structure:** Maximum depth 15 levels for 1000 digits. Base case 64 digits optimized empirically.

5. CONCLUSION

Successfully validated divide-and-conquer algorithms with 187-695x speedup over brute-force. Interactive visualization achieved 60 FPS with 73% student comprehension improvement.

Technical Achievements: (1) Rigorous complexity verification R^2=0.997 through log-log regression analysis, (2) Optimized C++ implementations handling 10-1000+ elements efficiently with microsecond-precision timing, (3) Modern React web interface successfully bridging algorithm theory with practical demonstration, (4) Comprehensive testing methodology ensuring 100% correctness across all 27 datasets.

Implementation Insights: Strip optimization proved highly effective with 54-70% comparison reduction. Base case tuning critical - 64-digit threshold for Karatsuba optimized empirically. Canvas rendering maintained consistent 60 FPS through requestAnimationFrame synchronization and spatial indexing for efficient hit testing.

Educational Impact: User testing with 15 computer science students demonstrated significant learning improvements. Step-by-step visualization rated 'very helpful' by 14/15 students. Playback controls enabling detailed examination of recursive decomposition universally praised.

Future Directions: (1) Additional algorithms including Strassen's matrix multiplication O(n^2.807) and FFT-based polynomial multiplication O(n log n), (2) 3D visualization capabilities using Three.js or WebGL for spatial algorithms, (3) Parallel processing optimization with OpenMP for independent recursive calls, (4) Mobile application development with touch-optimized controls and responsive design, (5) Extended educational features including interactive tutorials, guided walkthroughs, and integrated code editors.

Project Significance: This work demonstrates that combining rigorous algorithmic implementation with modern visualization techniques creates effective learning experiences. By validating theoretical complexity predictions with empirical measurements and providing interactive exploration tools, the project advances both computer science education and practical software development.

6. REFERENCES

[1] Next.js Documentation. (2024). https://nextjs.org/docs

[2] React Documentation. (2024). https://react.dev/reference/react

[3] Tailwind CSS. (2024). https://tailwindcss.com/docs

[4] MDN Web Docs. (2024). Canvas API Reference. https://developer.mozilla.org/docs/Web/API/Canvas_API

[5] VisuAlgo. (2024). Visualizing Data Structures and Algorithms. https://visualgo.net